INSIDE STLC ACADEMY

PRACTICAL TEST AUTOMATION WITH PLAYWRIGHT

Page Object Model (POM) Template

Structure tests for reuse and maintainability with page objects.

Introduction

The Page Object Model keeps locators and actions for a page in one place, so tests read clearly and changes happen in one spot. This template gives you a clean, reusable structure.

Instructions

- Create one class per page/component.
- Keep locators and actions inside the class.
- Keep assertions in the test (or expose verify methods).

Worked Example (LoginPage)

- class LoginPage {
- constructor(page) { this.page = page; }
- emailInput = () => this.page.getByLabel('Email');
- signInButton = () => this.page.getByRole('button', { name: 'Sign in' });
- async login(email, password) { /* fill + click */ }
- }

How the Test Reads

const login = new LoginPage(page); await login.goto(); await login.login(user, pass); await expect(page.getByRole('heading', { name: 'Dashboard' })).toBeVisible();

Blank Reusable Template

- Page class name:
- URL / goto():
- Key locators:
- Actions (methods):
- What the test asserts:

Practical Exercise

Refactor one of your existing tests to use a page object. Notice how much cleaner the test reads.

Best Practices

- One page object per page/component.
- Expose intent-revealing methods (login, addToCart).
- Keep assertions out of locators-only objects.

Common Mistakes

- Over-engineering POM for a tiny project.
- Putting test logic/assertions inside page objects.
- Duplicating locators across files instead of reusing the object.

> **INSIDE STLC PRO TIP**
>
> Start simple: introduce page objects when you find yourself repeating locators. Premature abstraction is as harmful as none — let the duplication tell you when.